



Audyogik Shikshan Mandals Nextgen Technical Campus

Survey No. 401/1, Adjacent to Parshwa Prandyalay Jain Temple Talegaon Dabhade,
Maval, Pune -410 506 www.asmnex.edu.in / Email : nextg@asmedu.org
Affiliated to SPPU and Approved by AICTE

5. Tool Support for Testing

- 5.1. Types of Test tools –CAST (only type & their purpose & Benefits and Risks should be covered)
- 5.2. Introduction of a tool into an organization
- 5.3. Testing tools
 - 5.3.1. Selenium -WebDriver and Test NG
 - 5.3.2. JMeter
 - 5.3.3. Postman
 - 5.3.4. ETL Testing Tool
- 5.4. JIRA (Project Management)

5.1. Types of Test tools –CAST (only type & their purpose & Benefits and Risks should be covered)

Computer-Aided Software Testing (CAST) tools are software applications that help testers automate and manage different testing activities. These tools make testing faster, more accurate, and more efficient by reducing manual effort.

CAST tools are used for various testing tasks such as:

- Managing test plans and cases
- Automating test execution
- Checking code for errors without running it
- Testing system performance under different conditions
- Identifying security vulnerabilities
- Tracking and managing software defects

By using these tools, software teams can improve test quality, find defects early, and ensure that software works as expected. However, using CAST tools also comes with challenges, such as learning how to use them effectively and maintaining them over time.

1. Test Management Tools

Purpose:

- Help plan, track, and manage testing activities
- Store test cases, execution results, and reports.
- Allow collaboration among testers and developers.

Benefits:

- Ensures organized and efficient test management.
- Provides traceability from requirements to test cases.
- Helps in tracking defects and test coverage.

Risks:

- Can be complex and time-consuming to set up.
- May require integration with other tools.

Examples: TestRail, qTest, HP ALM

2. Test Execution Tools (Automation Tools)

Purpose:

- Automate the execution of test cases to speed up testing.
- Compare actual vs. expected results automatically.
- Reduce manual errors.

Benefits:

- Increases testing speed and accuracy.
- Allows repeated execution of tests.
- Saves manual effort.

Risks:

- High initial setup and maintenance cost.
- Requires programming skills.
- May fail if test scripts are not updated when the application changes.

Examples: Selenium, TestComplete, Katalon Studio

3. Static Testing Tools

Purpose:

- Analyze code, documents, and models without executing them.
- Identify coding errors, security flaws, and maintainability issues.

Benefits:

- Detects issues early in development.
- Improves code quality and security.

Risks:

- May generate false warnings.
- Requires expertise to analyze results correctly.

Examples: SonarQube, Checkmarx, ESLint

4. Test Design Tools

Purpose:

- Help testers create and manage test cases.
- Generate test cases based on models, requirements, or risk analysis.

Benefits:

- Ensures systematic test coverage.
- Reduces redundancy in test case creation.

Risks:

- May not fully capture real-world scenarios.
- Some tools have a learning curve.

Examples: TestLink, TOSCA, SmartBear TestComplete

5. Performance Testing Tools

Purpose:

- Simulate different levels of load on an application.
- Identify performance issues like slow response time and system crashes.

Benefits:

- Helps optimize system performance.
- Ensures the application can handle high user traffic.

Risks:

- Requires a realistic test environment.
- Can be resource-intensive.

Examples: JMeter, LoadRunner, Gatling

6. Security Testing Tools

Purpose:

- Identify security vulnerabilities such as SQL injection, XSS, and weak authentication.
- Help ensure data protection and compliance.

Benefits:

- Protects applications from cyber threats.
- Helps meet security standards.

Risks:

- May generate false positives.
- Requires security expertise to interpret findings.

Examples: OWASP ZAP, Burp Suite, Nessus

7. Defect Tracking Tools

Purpose:

- Log, track, and manage software defects.
- Assign bugs to developers and monitor their resolution.

Benefits:

- Provides a centralized system for tracking defects.
- Improves communication between teams.

Risks:

- Can be complex if not managed properly.
- May slow down workflow if overloaded with unnecessary data.

Examples: Jira, Bugzilla, Redmine

1. **Test Management Tools** – For planning, tracking, and managing test activities.
2. **Test Execution Tools (Automation Tools)** – For automating the execution of test cases.
3. **Static Testing Tools** – For analyzing code and documents without execution.
4. **Test Design Tools** – For creating and managing test cases.
5. **Performance Testing Tools** – For checking system performance under different conditions.
6. **Security Testing Tools** – For finding security vulnerabilities.
7. **Defect Tracking Tools** – For logging and managing software defects

5.2. Introduction of a tool into an organization

The introduction of a tool in an organization refers to the process of selecting, implementing, and integrating a new software or system to improve efficiency, productivity, or quality of work.

Key Steps Involved

1. Here are your points with **1–2 extra lines added** (simple and exam-ready):
2. **Identify the Need :** First, the organization identifies a problem or requirement where the tool can help, such as improving testing, communication, or project management. This step ensures that the tool is actually needed and will solve a real problem. It helps avoid unnecessary investment.
3. **Tool Selection :** Different tools are evaluated based on cost, features, compatibility, and ease of use. The best-suited tool is selected. Comparison of multiple tools helps in making the right decision. Proper selection increases efficiency and reduces future issues.
4. **Planning and Preparation :** A proper plan is created for implementation, including budget, timeline, and resources required. Roles and responsibilities are also clearly defined. This ensures smooth execution without confusion.
5. **Installation and Setup:** The tool is installed and configured according to the organization's needs. System requirements and settings are properly adjusted. This step makes the tool ready for actual use.
6. **Training Employees :** Staff members are trained so they can use the tool effectively and confidently. Training sessions, demos, or manuals may be provided. It reduces errors and increases productivity.
7. **Pilot Testing :** The tool is tested on a small scale to check if it works properly before full implementation. Any issues or bugs are identified and fixed at this stage. It reduces risk before full deployment.
8. **Full Implementation:** After successful testing, the tool is deployed across the organization. All users start using the tool in a real work environment. This step marks complete adoption of the tool.
9. **Monitoring and Support :** Performance is monitored, and technical support is provided to resolve issues. Regular updates and maintenance are also carried out. It ensures long-term smooth functioning of the tool.

5.3. Testing tools

5.3.1. Selenium -WebDriver and Test NG

Selenium is one of the most popular open-source **automation testing tools** used for testing web applications. It allows testers to automate web interactions and check if websites function correctly on different browsers like **Chrome, Firefox, Edge, and Safari**.

Selenium consists of different components:

1. **Selenium WebDriver** : Automates web browsers by controlling them programmatically. It allows writing test scripts in languages like Java, Python, etc. It interacts with browser elements just like a real user (click, type, navigate).
2. **Selenium IDE** : A record-and-playback tool for quick test creation. It records user actions in the browser and converts them into test scripts. It is easy to use and suitable for beginners and simple testing tasks.
3. **Selenium Grid** : Runs tests in parallel on different machines and browsers. It helps save time by executing multiple test cases at the same time. It is useful for cross-browser and distributed testing environments.

Among these, **Selenium WebDriver** is the most widely used because it is flexible, supports multiple programming languages (**Java, Python, C#, etc.**), and integrates well with test frameworks like **TestNG**.

1. Selenium WebDriver

Selenium WebDriver is a tool used to control web browsers automatically. It helps testers perform actions like clicking, typing, and opening websites. You can write instructions using simple programming languages like Java or Python. It works with different browsers like Chrome and Firefox.

It is used to test websites quickly without doing everything manually. Overall, it makes testing faster and easier. It also reduces human errors during testing. It is very useful for repetitive tasks like login testing.

- Open a webpage
- Click buttons and links
- Enter text in input fields
- Validate if a webpage behaves as expected

Features of Selenium WebDriver

- Supports multiple browsers (Chrome, Firefox, Edge, Safari)
- Works with different programming languages (Java, Python, C#)
- Can handle dynamic web elements using XPath and CSS selectors
- Supports headless browser testing (running tests without opening a browser window)

Example

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class SeleniumExample {

    public static void main(String[] args) { // Set the path of the ChromeDriver executable
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");

        // Launch Chrome browser
        WebDriver driver = new ChromeDriver();

        // Open Google
        driver.get("https://www.google.com");

        // Print the title of the webpage
        System.out.println("Title: " + driver.getTitle());

        // Close the browser
        driver.quit();
    }
}
```

Advantages of Selenium WebDriver

- Open-source and free to use
- Cross-browser compatibility
- Supports mobile testing using Appium
- Can handle alerts, pop-ups, and frames

Challenges of Selenium WebDriver

- Requires programming knowledge
- Cannot generate test reports automatically
- Does not support desktop or mobile app testing (only web applications)

2. TestNG (Test Next Generation)

TestNG (Test Next Generation) is a powerful testing framework inspired by JUnit and NUnit, used for automation testing. It supports features like annotations, parallel execution, test grouping, and prioritization to manage test cases effectively.

TestNG allows data-driven testing using DataProvider and supports dependency between test methods. It is commonly used with Selenium WebDriver for web application testing. The framework also generates detailed HTML reports for test results. Overall, TestNG improves flexibility, efficiency, and control in test execution.

Why Use TestNG?

- Allows **grouping and prioritizing** test cases
- Supports **parallel execution** (running multiple tests at the same time)
- Generates **detailed test reports**
- Provides **annotations** (`@Test`, `@BeforeMethod`, `@AfterMethod`) for better test management

Example: TestNG Script in Java

This program prints a message when the test runs:

```
import org.testng.annotations.Test;

public class TestNGExample {

    @Test

    public void testMethod() {

        System.out.println("TestNG test executed successfully");

    }

}
```

Advantages of TestNG

- Provides test reports automatically
- Easier test execution management
- Supports parallel test execution

- Allows data-driven testing using XML files

Challenges of TestNG

- Requires XML configuration for advanced features
- Can be complex for beginners

Integrating Selenium WebDriver with TestNG

Selenium WebDriver alone does not generate test reports or support test case management. By integrating **Selenium WebDriver with TestNG**, we can:

- Automate web testing efficiently
- Run multiple test cases easily
- Get detailed test reports

TestNG Annotations

Annotation	Description
@Test	Marks a method as a test case(This is the most important annotation. It tells TestNG that the method is a test case.)
@BeforeMethod	This annotation is used to set up things before each test case runs .
@AfterMethod	Cleans up resources after each test case runs .
@BeforeClass	Runs only once before any test method in the class. Used for setup like opening a browser.
@AfterClass	Runs only once after all test cases are executed.

5.3.2. JMeter

JMeter is an open-source tool used for **testing the performance** of web applications and services. It helps check how well a website, server, or application performs when **many users** are using it at the same time.

Developed by **Apache Software Foundation**.

What Can JMeter Do?

- Test websites (HTTP, HTTPS).

- Test web services (SOAP, REST APIs).
- Test databases (JDBC).
- Simulate multiple users to check how the system behaves under load (Load Testing).
- Measure response time, throughput, and errors.

Why Use JMeter?

- Free and open-source.
- Supports multiple types of tests (load, stress, functional, performance).
- Easy to use with a graphical interface.
- Can simulate hundreds or thousands of virtual users.
- Supports report generation and test automation.

Basic Components of JMeter

1. **Test Plan** – The overall test structure.
2. **Thread Group** – Represents a group of virtual users.
3. **Sampler** – Sends requests to the server (like HTTP request).
4. **Listener** – Shows the result of the test (like graphs, tables).
5. **Assertions** – Checks if the response is correct (like status = 200).
6. **Timers** – Adds delays between requests.

Types of Testing with JMeter

- **Load Testing** – To check performance under expected user load.
- **Stress Testing** – To test the limit of the system.
- **Spike Testing** – To test performance when the number of users suddenly increases.
- **Endurance Testing** – To see if the system can handle continuous load over time.

How JMeter Works (Step-by-Step):

1. You create a Test Plan with multiple Thread Groups.
2. Each Thread Group simulates multiple users.

3. Samplers send requests (like users visiting a website).
4. Listeners collect and display the results.

5.3.3. Postman

Postman is a popular tool used to test APIs (Application Programming Interfaces). It helps developers and testers to send requests to APIs and check the responses. Postman provides a user-friendly interface to test how your application communicates over the internet.

You can easily send different types of requests like GET, POST, PUT, and DELETE. It also allows saving and organizing requests for future use. Postman helps in finding errors quickly and improves API testing efficiency.

What is an API?

- API stands for Application Programming Interface.
- It allows two software applications to communicate with each other.
- For example, when you log in to an app using your Google account, it uses an API to connect to Google.

Why Use a Postman?

- To test APIs easily without writing code.
- To check if the API is working correctly.
- To send different types of requests (GET, POST, PUT, DELETE).
- To see API responses, status codes, and time taken.
- Helps with manual testing and automation of API tests.

Features of Postman:

1. **Simple Interface** – Easy to use with buttons and input fields.
2. **Request Types** – Supports GET, POST, PUT, DELETE, PATCH, etc.
3. **Collections** – Save and organize API requests.
4. **Environments** – Store variables like API keys and tokens.
5. **Pre-request Scripts** – Write JavaScript before sending a request.
6. **Tests (Assertions)** – Write test cases to validate response data.

7. **History** – Keeps track of your previous requests.
8. **Automation** – Use Collection Runner to run multiple tests automatically.

Common HTTP Methods in Postman:

Method	Purpose
GET	Read data from the server
POST	Send data to the server
PUT	Update existing data
DELETE	Remove data from the server

How to Use Postman – Basic Steps:

1. Open Postman.
2. Select the HTTP method (e.g., GET, POST).
3. Enter the API URL.
4. Add parameters, headers, or body data if needed.
5. Click Send.
6. View the response, including status code, time, and data.

Benefits of Using Postman:

- Makes API testing easy and fast.
- Saves time in **manual testing**.
- Useful for both **developers** and **testers**.
Works on **Windows, Mac, and Linux**.
- Allows **team collaboration** using Postman Cloud.

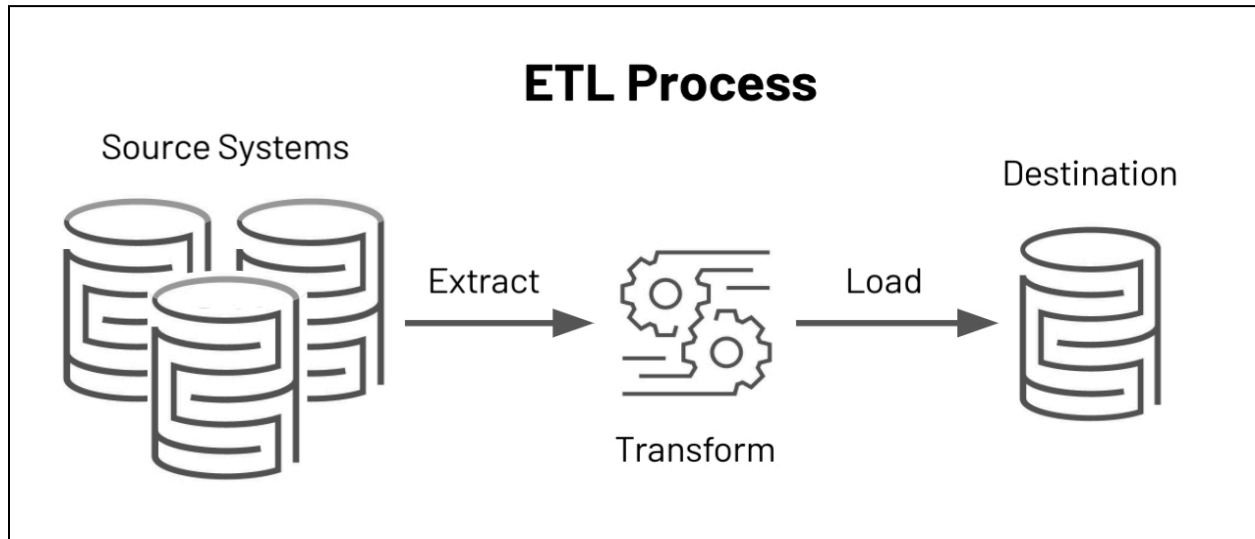
5.3.4. ETL Testing Tool

ETL stands for:

- **E – Extract** data from different sources (like databases, files, APIs).
- **T – Transform** the data into the required format (cleaning, filtering, converting).

- **L – Load** the data into a target system (like a data warehouse or database).

ETL is used in **data warehousing** and **business intelligence** to make data useful for reporting and analysis.



ETL Testing is the process of:

- Checking the accuracy of the data moved from source to target.
- Making sure the transformation logic is applied correctly.
- Verifying that no data is lost or duplicated.
- Ensuring data quality, performance, and completeness.

What is an ETL Testing Tool?

An **ETL Testing Tool** is a software that helps testers:

- Automate the process of testing ETL jobs.
- Validate data at different stages (before and after transformation).
- Compare source and target data.
- Create test cases and reports for ETL workflows.

Example Scenario:

A company extracts customer data from a CRM, transforms it to match reporting format, and loads it

into a data warehouse.

ETL testing ensures that all customer records are correctly processed and available for reports.

Types of ETL Testing:

1. **Data Validation Testing** – Check if data is correctly loaded from source to target.
2. **Data Transformation Testing** – Verify that transformation logic is working as expected.
3. **Data Completeness Testing** – Ensure no data is lost.
4. **Data Accuracy Testing** – Check that calculations, joins, filters, etc., are done correctly.
5. **Performance Testing** – Check how fast ETL jobs are running.
6. **Regression Testing** – Re-run test cases after changes in ETL code.

5.4. JIRA (Project Management)

JIRA is a project management and issue tracking tool developed by Atlassian. It is widely used in software development to plan, track, and manage projects efficiently. Teams use JIRA to create and manage tasks such as bugs, user stories, and work items.

JIRA supports agile methodologies like Scrum and Kanban, helping teams organize work using boards and sprints. It allows assigning tasks to team members, setting priorities, and tracking progress in real time. The tool also provides reports and dashboards to monitor project performance. Overall, JIRA improves team collaboration, transparency, and project management efficiency.

Who Uses JIRA?

- Software Developers
- Testers (QA)
- Project Managers
- Scrum Masters
- Agile Teams

Key Features of JIRA:

1. **Issue Tracking** – Report and manage bugs, tasks, and enhancements.
2. **Project Management** – Organize work using boards, sprints, and backlogs.

3. **Custom Workflows** – Set up different workflows for different project needs.
4. **Agile Support** – Supports **Scrum** and **Kanban** boards.
5. **Reports and Dashboards** – Track progress with visual charts and reports.
6. **Collaboration** – Assign tasks, add comments, attach files, and tag team members.

Important JIRA Terms:

Term	Meaning
Issue	A task, bug, story, or request that needs to be tracked.
Project	A collection of issues and tasks under a team or product.
Epic	A large body of work that can be broken down into smaller tasks.
Story	A user requirement or feature.
Task	A piece of work to be done.
Bug	A problem or defect in the system.
Sprint	A short time period (e.g., 2 weeks) in Agile to complete set tasks.
Backlog	A list of all tasks or stories to be worked on.

How JIRA is Used in Project Management:

1. **Create a Project** – Define the scope and goal of the project.
2. **Add Issues** – Add tasks, bugs, or user stories.
3. **Assign Issues** – Assign work to team members.
4. **Track Progress** – Use boards to see which tasks are “To Do,” “In Progress,” or “Done”.
5. **Use Sprints** – Divide work into time-boxed periods (Scrum).
6. **Monitor and Report** – Use dashboards and charts to track performance.

Example Use Case:

A software development team is building an e-commerce website.
They use JIRA to:

- Create a project for the website
- Add user stories like "Add to Cart" and "Checkout"
- Track bugs like “Login not working”
- Assign tasks to developers and testers
- Run 2-week sprints to finish features

Difference

Point	Selenium	Apache JMeter	Postman	JIRA
1	Automates web browsers	Used for performance testing	Used for API testing	Used for project management
2	Tests web applications	Tests system under load	Sends API requests	Tracks bugs and issues
3	Supports Java, Python	Simulates multiple users	Supports GET, POST, etc.	Manages tasks/stories
4	Works on Chrome, Firefox	Tests web apps & APIs	Checks API responses	Supports Agile methods
5	Performs user actions	Finds performance issues	User-friendly interface	Assigns tasks
6	Functional testing	Provides reports/graphs	Saves requests	Tracks progress
7	Cross-browser testing	Open-source tool	Supports automation	Provides dashboards
8	Reduces manual work	Ensures system stability	Finds errors quickly	Improves collaboration